开篇

NoteKing | 图文讲义

Author / Channel	MiniMind / Mokio
Duration	2m50s
Generated by	NoteKing — Video to Illustrated Notes
Date	2026-04-03
Video Link	https://www.bilibili.com/video/BV1T2k6BaEeC
GitHub	github.com/bcefghj/noteking
Xiaohongshu	bcefghj

目录

1 课程概述 **2**

2 为什么学习这门课程？ **2**

2.1 大语言模型的崛起 2

2.2 课程目标 3

3 课程大纲预览 **4**

3.1 第一阶段：基础与核心（第 1-5 章） 4

3.2 第二阶段：模型实现（第 6-12 章） 5

3.3 第三阶段：训练与优化（第 13-20 章） 6

3.4 第四阶段：部署与应用（第 21-26 章） 7

4 工具与环境准备 **8**

4.1 推荐的开发环境 8

4.2 硬件要求 8

5 数学基础回顾 **9**

5.1 神经网络核心数学 9

5.1.1 1. 矩阵运算 9

5.1.2 2. Softmax 函数 9

5.1.3 3. 交叉熵损失 9

5.1.4 4. 链式法则 10

6 PyTorch 快速入门 **10**

6.1 张量基础 10

6.2 自动求梯度 11

7 课程学习方法 **11**

7.1 推荐的学习路径 11

7.2 代码实践要点 11

7.3 遇到问题怎么办? 12

8 本章小结 **13**

9 关键要点（Key Takeaways） **13**

10 思考题 **14**

第 1 集：开篇

MiniMind Episode 1/26：开篇—课程介绍与背景

1 课程概述

MINI 课程标题：MiniMind - PyTorch 从零手敲大模型/MINI

MINI 时长：2 分 50 秒/MINI

MINI 难度：入门级/MINI

欢迎来到 **MiniMind** 系列课程！本课程将从零开始，手把手带领大家使用 PyTorch 框架构建大型语言模型（Large Language Model, LLM）。无论你是深度学习的初学者，还是希望深入理解大模型原理的开发者，本课程都将为你提供系统、完整的知识体系。

图 1：Video 00:03 [1]

2 为什么学习这门课程？

2.1 大语言模型的崛起

过去几年，生成式 AI 技术取得了突破性进展。以 ChatGPT、Claude、GPT-4 为代表的对话系统展现了令人惊叹的语言理解和生成能力。这些技术的核心正是 **大型语言模型（LLM）**。

背景知识

大型语言模型是基于 Transformer 架构的深度神经网络，通过在大规模文本语料上进行预训练，学习语言的统计规律和语义表示。其参数量从数亿到数千亿不等，规模的增大带来了能力的质变。

[1] video timestamp 00:03

2.2 课程目标

图 2: Video 00:13 [2]

本课程的核心目标是：

1. **理解原理**：深入理解 Transformer、注意力机制、位置编码等核心概念
2. **动手实现**：从零开始用 PyTorch 实现完整的大模型训练流程
3. **实践应用**：完成预训练、有监督微调、RLHF 等关键步骤
4. **培养能力**：建立对大模型全貌的认知，能够独立探索前沿技术

核心概念

为什么选择”从零手敲”？

>

市面上已有许多使用现成库（如 *Hugging Face Transformers*）的教程。但”手敲”能让你真正理解每一步的实现细节，而不是盲目调用 *API*。这是成为 *AI* 工程师的必经之路。

[2] video timestamp 00:13

3 课程大纲预览

图 3: Video 00:23 [3]

本系列共 **26 个章节**，涵盖以下核心内容：

3.1 第一阶段：基础与核心（第 1-5 章）

章节	内容	关键知识点
1	课程介绍	课程目标、学习路径
2	PyTorch 基础	张量、自动微分
3	Transformer 架构	编码器-解码器结构
4	注意力机制	Self-Attention、Multi-Head Attention
5	位置编码	绝对位置编码、相对位置编码

[3] video timestamp 00:23

3.2 第二阶段：模型实现（第 6-12 章）

图 4: Video 00:33 [4]

- MiniMind 模型架构设计
- Tokenizer（分词器）实现
- 词嵌入与输出层
- 训练循环构建
- 分布式训练基础

[4] video timestamp 00:33

3.3 第三阶段：训练与优化（第 13-20 章）

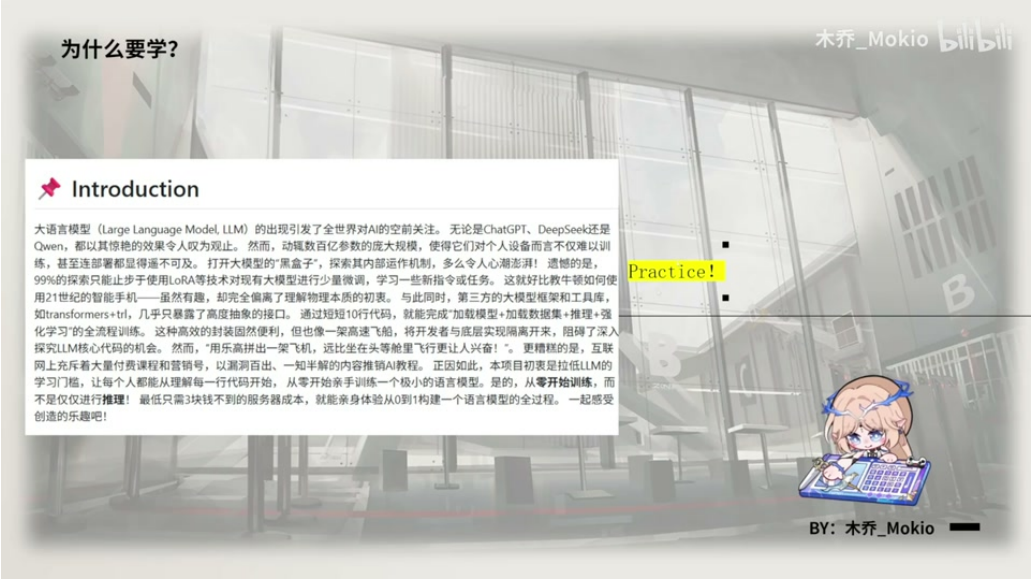

图 5: Video 00:43 [5]

- 预训练（Pretraining）
- 有监督微调（Supervised Fine-tuning, SFT）
- 人类反馈强化学习（RLHF）
- 偏好对齐技术（DPO、PPO）
- 训练技巧与调优

[5] video timestamp 00:43

3.4 第四阶段：部署与应用（第 21-26 章）

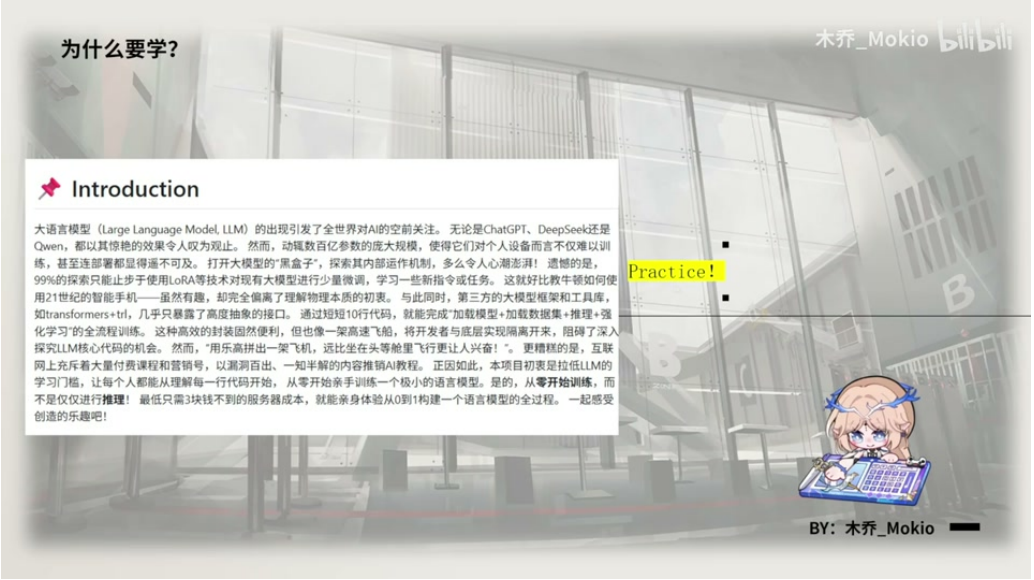

图 6: Video 00:53 [6]

- 模型压缩与量化
- 推理优化
- API 服务部署
- 前端应用集成
- 项目实战

[6] video timestamp 00:53

4 工具与环境准备

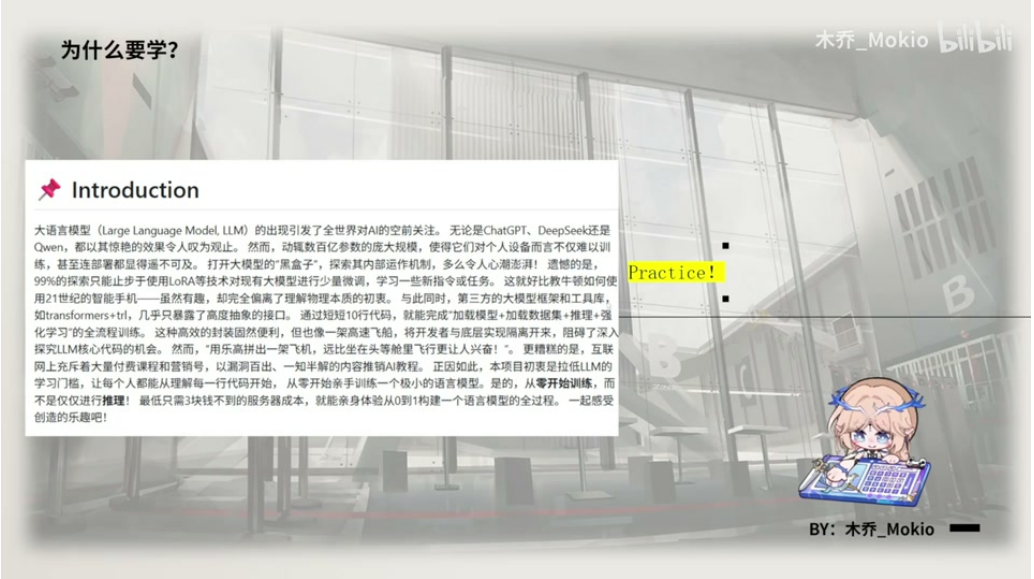

图 7: Video 01:03 [7]

4.1 推荐的开发环境

背景知识

```python
# Python 版本要求
# 推荐使用 Python 3.10 或更高版本

import sys
print(f"Python 版本: {sys.version}")

# 核心依赖包
# pip install torch>=2.0.0
# pip install numpy
# pip install transformers  # 辅助学习用
# pip install wandb         # 训练可视化（可选）
# pip install tensorboard   # 训练日志（可选）
```

4.2 硬件要求

配置级别	GPU 需求	适用场景
入门级	NVIDIA GTX 1060 (6GB)	学习基本概念、跑小模型
标准级	NVIDIA RTX 3090 (24GB)	完成课程大部分实验
高端级	NVIDIA A100 (40GB+)	训练更大规模模型

[7] video timestamp 01:03

易错点

***GPU* 显存不足的常见原因：**

>

1. Batch size 设置过大

2. 未使用混合精度训练（*Mixed Precision*）

3. 模型参数未及时释放

>

初学者建议从较小的模型和 *batch size* 开始，逐步增大。

---

5 数学基础回顾

5.1 神经网络核心数学

本课程涉及以下数学知识，建议提前复习：

5.1.1 1. 矩阵运算

$$Y = X \cdot W + b$$

其中 X 为输入矩阵，W 为权重矩阵，b 为偏置向量。

5.1.2 2. Softmax 函数

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

用于将任意实数向量转换为概率分布。

5.1.3 3. 交叉熵损失

$$\mathcal{L} = -\sum_i y_i \cdot \log(\hat{y}_i)$$

衡量预测分布与真实分布之间的差异，是分类任务常用的损失函数。

5.1.4 4. 链式法则

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \{} \cdot \frac{\partial \{}{\partial \theta}$$

反向传播算法的数学基础。

背景知识

***PyTorch* 如何自动求导？**

>

PyTorch 通过 `autograd` 模块自动构建计算图。当你执行前向传播时，*PyTorch* 会记录所有操作；调用 `.backward()` 时，它会根据链式法则自动计算所有参数的梯度。

---

6 PyTorch 快速入门

6.1 张量基础

张量（Tensor）是 PyTorch 中的核心数据结构，可以理解为多维数组。

```python
import torch

# 创建张量的几种方式
# 1. 从 Python 列表创建
x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)

# 2. 随机初始化
w = torch.randn(512, 768)  # 正态分布随机
b = torch.zeros(512)       # 全零初始化

# 3. 从 numpy 数组转换
import numpy as np
np_array = np.array([[1, 2], [3, 4]])
x = torch.from_numpy(np_array).float()

# 张量基本操作
print(f"形状: {x.shape}")
print(f"数据类型: {x.dtype}")
print(f"设备: {x.device}")
```

6.2 自动求梯度

```
# 开启梯度追踪
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# 定义计算图
y = x ** 2
z = y.mean()

# 反向传播，计算梯度
z.backward()

# 查看梯度
print(x.grad)  # 输出：tensor([0.6667, 1.3333, 2.0000])
```

核心概念

计算图（*Computational Graph*）

>

PyTorch 使用动态计算图（*Dynamic Computational Graph*）。每次前向传播时，图会被实时构建，这使得调试更加直观，也方便使用 *Python* 的控制流语句。

---

7 课程学习方法

7.1 推荐的学习路径

1. **观看视频 → 2. 阅读讲义 → 3. 动手敲代码 → 4. 独立复现 → 5. 扩展实验**

7.2 代码实践要点

```
# 学习代码时的正确姿势

#   不要只是复制粘贴
model = Transformer()

#   要理解每一行代码的作用
class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model, nhead, num_layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
```

```python
        self.transformer = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(
                d_model=d_model,
                nhead=nhead
            ),
            num_layers=num_layers
        )
        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.transformer(x)
        x = self.fc(x)
        return x
```

7.3 遇到问题怎么办？

1. **查看官方文档**：PyTorch Documentation
2. **搜索引擎**：优先使用英文关键词
3. **GitHub Issues**：查找类似问题
4. **社区提问**：Stack Overflow、知乎等

易错点

调试时的常见错误：

>

“‘python

 # 错误：忘记将数据和模型移到同一设备

model = model.cuda()

output = model(input) # input 还在 CPU 上！

>

 # 正确做法

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = model.to(device)

input = input.to(device)

output = model(input)

“‘

8 本章小结

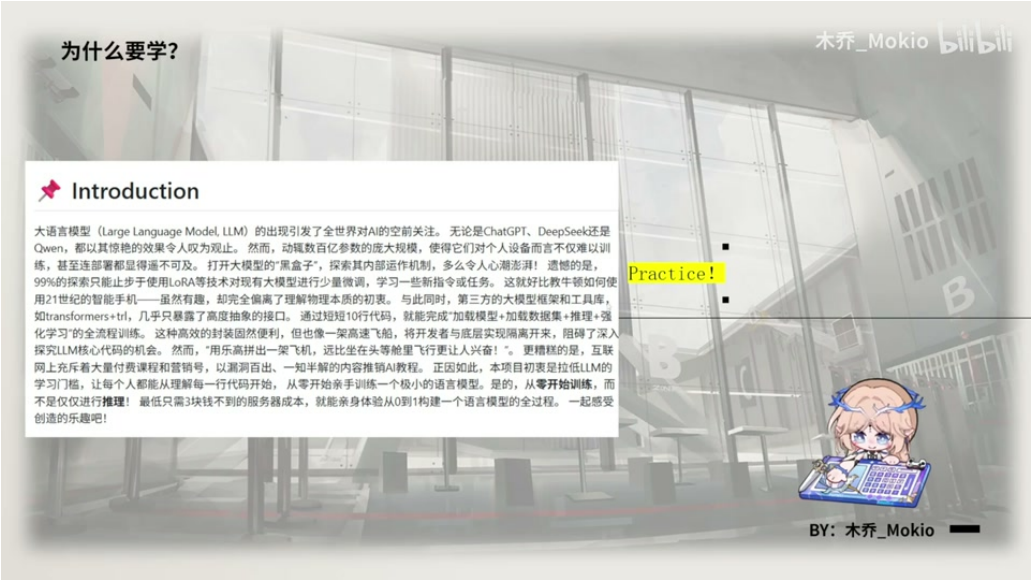

图 8: Video 01:03 [8]

本章作为课程的开篇，主要完成了以下内容：

知识点	掌握程度
课程目标与大纲	理解整体学习路径
大模型基本概念	了解 LLM 的重要性
PyTorch 环境配置	能够搭建开发环境
张量与自动求导基础	掌握核心 API 用法
数学基础	复习关键公式

9 关键要点（Key Takeaways）

必记住的核心内容

1. **大型语言模型**是基于 Transformer 架构的深度学习模型，能够理解和生成人类语言

[8] video timestamp 01:03

2. **PyTorch** 使用动态计算图和 `autograd` 模块实现自动微分
3. **从零手敲**是理解大模型原理的最佳方式，优于直接调用高层 API
4. **课程涵盖 26 个章节**，从基础概念到部署应用，体系完整

---

10 思考题

1. **为什么大模型的"大"如此重要？**思考模型规模（参数量）与能力之间的关系，以及是否存在规模化的极限。
2. **动态计算图 vs 静态计算图**：PyTorch 使用动态图，TensorFlow 使用静态图（如 TF1）。这两种方式的优缺点分别是什么？为什么后来 TensorFlow 也引入了 Eager Execution?
3. **实践挑战**：如果你只有一块 8GB 显存的 GPU，想训练一个 7B 参数的模型，可能面临哪些挑战？有哪些解决方案?